

Adama sarr

Correction Atelier Numpy

Contexte

Une entreprise dispose de plusieurs capteurs installés dans différents bâtiments. Les capteurs mesurent régulièrement la température, l'humidité, la pression et la consommation énergétique.

Les données doivent être préparées et analysées avant d'être transmises à un futur système de Machine Learning permettant de détecter des situations anormales

Partie 1 : Listes Python vs tableaux NumPy

On dispose des températures mesurées par un capteur : 24.5, 25.1, 26.3, 27.0 et 25.8

- 1) Créer une liste Python stockant ces températures ;

```
temperature = [24.5, 25.1, 26.3, 27.0, 25.8]
print("Temperature:", temperature)
```

[3] ✓ 0.0s Python

... Temperature: [24.5, 25.1, 26.3, 27.0, 25.8]

- 2) Créer un tableau NumPy équivalent stockant ces températures ;

```
temperature = np.array([24.5, 25.1, 26.3, 27.0, 25.8])
print("Temperature:", temperature)
```

[8] ✓ 0.0s Python

... array([24.5, 25.1, 26.3, 27.0, 25.8])

- 3) Effectuer une multiplication par 2 sur ces deux collections ;

```
# multiplication par 2 pour la liste
temperature_X2 = temperature * 2
print("Temperature_X2:", temperature_X2)
```

[4] ✓ 0.0s Python

... [24.5, 25.1, 26.3, 27.0, 25.8, 24.5, 25.1, 26.3, 27.0, 25.8]

```
# Multiplication par 2 pour le tableau numpy
temperature_tab_X2 = temperature * 2
print("Temperature_tab_X2:", temperature_tab_X2)
```

[6] ✓ 0.0s Python

... array([49.0, 50.2, 52.6, 54.0, 51.6])

- 4) Quelle différence observe-t-on ?

Lorsqu'on multiplie une liste Python par 2, les éléments de la liste sont répétés deux fois.

En revanche, lorsqu'on multiplie un tableau NumPy par 2, chaque température est multipliée par 2.

Ainsi, la différence est que la liste Python répète ses éléments, tandis que NumPy effectue une opération mathématique sur chaque élément du tableau.

Partie 2 – Création des données

- 1) Créer un tableau représentant les mesures de température de 10 capteurs, initialement à zéro.

```
# Températures de 10 capteurs, initialement à zéro
temp_10_capteurs = np.zeros(10)
print(temp_10_capteurs)
```

[22] ✓ 0.0s Python

... [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]

- 2) Créer un tableau contenant 10 valeurs égales à 1.

```
# Tableau de 10 valeurs egales à 1
temp_10_capteurs_1 = np.ones(10)
print(temp_10_capteurs_1)
```

✓ 0.0s Python

[1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]

- 3) Créer un tableau de 10 valeurs initialisées à 25.

```
# Tableau de 10 valeurs initialisées à 25
temp_10_capteurs_25 = np.full(10, 25)
print(temp_10_capteurs_25)
```

✓ 0.0s Python

[25 25 25 25 25 25 25 25 25 25]

- 4) Créer une matrice identité de taille 5×5 .

```
# creation de la matrice identité 5x5
matrice_identite = np.eye(5)
print(matrice_identite)
```

✓ 0.0s Python

[[1. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 1.]]

- 5) Créer les 24 heures (de 0 à 23) de mesure d'une journée

```
# creation des 24 heures (de 0 à 23) de mesure d'une journée
heures = np.arange(24)
print(heures)
```

✓ 0.0s Python

[0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23]

- 6) Créer 10 valeurs régulièrement réparties entre 20 °C et 30 °C

```
# 10 valeurs régulièrement réparties entre 20 et 30
temp_10_capteurs_20_30 = np.linspace(20, 30, 10)
print(temp_10_capteurs_20_30)
```

✓ 0.0s Python

[20. 21.11111111 22.22222222 23.33333333 24.44444444 25.55555556
 26.66666667 27.77777778 28.88888889 30.]

- 7) Créer aléatoirement les températures de 100 mesures, en rendant les résultats reproductibles

```
# creation aleatoirement des temperatures de 100 mesures, rendant les resultats reproductibles
np.random.seed(42)
temp_100_capteurs = np.random.uniform(20, 30, 100)
print(temp_100_capteurs)
```

✓ 0.0s Python

[23.74540119 29.50714306 27.31993942 25.98658484 21.5601864 21.5599452
 20.58083612 28.66176146 26.01115012 27.08072578 20.20584494 29.69909852
 28.32442641 22.12339111 21.81824967 21.8340451 23.04242243 25.24756432
 24.31945019 22.9122914 26.11852895 21.39493861 22.92144649 23.66361843
 24.56069984 27.85175961 21.99673782 25.14234438 25.92414569 20.46450413
 26.07544852 21.70524124 20.65051593 29.48885537 29.65632033 28.08397348
 23.04613769 20.97672114 26.84233027 24.40152494 21.22038235 24.9517691
 20.34388521 29.09320402 22.58779982 26.62522284 23.11711076 25.20068021
 25.46710279 21.84854456 29.69584628 27.75132823 29.39498942 28.9482735
 25.97899979 29.21874235 20.88492502 21.95982862 20.45227289 23.25330331
 23.8867729 22.71349032 28.28737509 23.56753327 22.8093451 25.42696083
 21.40924225 28.02196981 20.74550644 29.86886937 27.72244769 21.98715682
 20.05522117 28.15461428 27.06857344 27.29007168 27.71270347 20.74044652
 23.58465729 21.1586906 28.63103426 26.23298127 23.30898025 20.6355835
 23.10982322 23.25183322 27.29606178 26.37557471 28.87212743 24.72214925
 21.19594246 27.13244787 27.60785049 25.61277198 27.7096718 24.93795596
 25.22732829 24.27541018 20.25419127 21.07891427]

Partie 3 – Exploration des tableaux

1) Créer aléatoirement un tableau représentant les 24 températures d'une journée :

```
np.random.seed(42)
temp_jour= np.random.uniform(24, 35, 24).round(1)
print(temp_jour)
```

```
[28.1 34.5 32.1 30.6 25.7 25.7 24.6 33.5 30.6 31.8 24.2 34.7 33.2 26.3
 26.  26.  27.3 29.8 28.8 27.2 30.7 25.5 27.2 28. ]
```

2) Déterminer les propriétés suivantes du tableau

a) Nombre de dimensions

```
# nbre de dimensions
print("Dimensions de temp_jour:", temp_jour.ndim)
```

```
Dimensions de temp_jour: 1
```

b) Nombre de valeurs

```
# nbre de valeurs
print("Nombre de valeurs dans temp_jour:", temp_jour.size)
```

```
Nombre de valeurs dans temp_jour: 24
```

c) Type des données

```
# le type des données
print("Type de données de temp_jour:", temp_jour.dtype)
```

```
Type de données de temp_jour: float64
```

d) Forme du tableau.

```
# format du tableau
print("Format du tableau temp_jour:", temp_jour.shape)
```

```
Format du tableau temp_jour: (24,)
```

Partie 4 – Indexation et slicing

Considérant le tableau représentant les 24 températures d'une journée, afficher :

1) la première mesure

```
# Affichons la première mesure du tableau temp_jour
print("Première mesure de temp_jour:", temp_jour[0])
```

```
Première mesure de temp_jour: 28.1
```

2) la dernière mesure

```
# Affichons la dernière mesure du tableau temp_jour
print("Dernière mesure de temp_jour:", temp_jour[-1])
```

```
Dernière mesure de temp_jour: 28.0
```

3) la température de 8h ;

```
# Affichons la température de 8 heure
print("Température de 8 heure:", temp_jour[7])
```

```
Température de 8 heure: 33.5
```

- 4) les températures de 8h à 12h

```
# Affichons les mesures de 8h à 12h du tableau temp_jour
print("Mesures de 8h à 12h:", temp_jour[7:12])
```

✓ 0.0s Python

Mesures de 8h à 12h: [33.5 30.6 31.8 24.2 34.7]

- 5) les températures des 6 premières heures ;

```
# Affichons les temperatures des 6 premieres heures
print("Mesures des 6 premières heures:", temp_jour[:6])
```

[52] ✓ 0.0s Python

... Mesures des 6 premières heures: [28.1 34.5 32.1 30.6 25.7 25.7]

- 6) les températures des heures paires.

```
# les temperatures des heures paires
print("Mesures des heures paires:", temp_jour[::2])
```

[54] ✓ 0.0s Python

```
... Mesures des heures paires: [28.1 32.1 25.7 24.6 30.6 24.2 33.2 26. 27.3 28.8 30.7 27.2]
```

Partie 5 – Filtrage booléen

Considérant le tableau représentant les 24 températures d'une journée, on souhaite détecter les situations inhabituelles.

- 1) Afficher les températures supérieures à 30 °C

```
# Affichons les temperature superieures à 30
print("Mesures supérieures à 30:", temp_jour[temp_jour > 30])
```

✓ 0.0s Python

Mesures supérieures à 30: [34.5 32.1 30.6 33.5 30.6 31.8 34.7 33.2 30.7]

- 2) Afficher les températures « normales » c'ad comprises entre 20 °C et 30 °C

```
# Affichons les temperatures normales (entre 20 et 30)
print("Mesures normales (entre 20 et 30):", temp_jour[(temp_jour >= 20) & (temp_jour <= 30)])
```

0.0s Python

```
Mesures normales (entre 20 et 30): [28.1 25.7 25.7 24.6 24.2 26.3 26. 26. 27.3 29.8 28.8 27.2 25.5 27.2
28. ]
```

- 3) Supposons qu'une température supérieure à 35 °C soit considérée comme anormale. Déterminer anomalies, un tableau de booléen précisant les anomalies

```
## Déterminons anomalies, un tableau de booléen précisant les anomalies
anomalies = temp_jour > 35
print("Anomalies (température > 35):", anomalies)
```

✓ 0.0s Python

```
Anomalies (température > 35): [False False False False False False False False False False False False
False False False False False False False False False False]
```

- 4) Déterminer le nombre de températures anormales à l'aide de la variable anomalies

```
# Determinons les nbre de temperatures anormales a l'aide de la variable anomalies
nbre_anomalies = np.sum(anomalies)
print("Nombre de mesures anormales (température > 35):", nbre_anomalies)
```

✓ 0.0s Python

Nombre de mesures anormales (température > 35): 0

Partie 6 – Manipulation des dimensions

On suppose qu'on a 5 capteurs.

- 1) Créer aléatoirement la matrice représentant les températures mesurées par ces capteurs durant une journée

```
# creation d'un matrice aleatoire representant les mesures de 5 capteurs sur 24 heures
np.random.seed(42)
mesures_5_capteurs = np.random.uniform(20, 30, (24, 5)).round(1)
print("Mesures de 5 capteurs sur 24 heures:\n", mesures_5_capteurs)
print("format", mesures_5_capteurs.shape)
```

✓ 0.0s Python

Mesures de 5 capteurs sur 24 heures:

```
[[23.7 29.5 27.3 26.  21.6]
 [21.6 20.6 28.7 26.  27.1]
 [20.2 29.7 28.3 22.1 21.8]
 [21.8 23.  25.2 24.3 22.9]
 [26.1 21.4 22.9 23.7 24.6]
 [27.9 22.  25.1 25.9 20.5]
 [26.1 21.7 20.7 29.5 20.7]
```

- 2) Transformer la matrice en un tableau unidimensionnel

```
# transformons la matrice en un tableau unidimensionnel
mesures_5_capteurs_flat = mesures_5_capteurs.flatten()
print("Mesures de 5 capteurs sur 24 heures (tableau unidimensionnel):\n", mesures_5_capteurs_flat)
print("format", mesures_5_capteurs_flat.shape)
```

✓ 0.0s Python

Mesures de 5 capteurs sur 24 heures (tableau unidimensionnel):

```
[23.7 29.5 27.3 26.  21.6 21.6 20.6 28.7 26.  27.1 20.2 29.7 28.3 22.1
 21.8 21.8 23.  25.2 24.3 22.9 26.1 21.4 22.9 23.7 24.6 27.9 22.  25.1
 25.9 20.5 26.1 21.7 20.7 29.5 29.7 28.1 23.  21.  26.8 24.4 21.2 25.
 20.3 29.1 22.6 26.6 23.1 25.2 25.5 21.8 29.7 27.8 29.4 28.9 26.  29.2
 20.9 22.  20.5 23.3 23.9 22.7 28.3 23.6 22.8 25.4 21.4 28.  20.7 29.9
 27.7 22.  20.1 28.2 27.1 27.3 27.7 20.7 23.6 21.2 28.6 26.2 23.3 20.6
 23.1 23.3 27.3 26.4 28.9 24.7 21.2 27.1 27.6 25.6 27.7 24.9 25.2 24.3
 20.3 21.1 20.3 26.4 23.1 25.1 29.1 22.5 24.1 27.6 22.3 20.8 22.9 21.6
 29.3 28.1 26.3 28.7 28.  21.9 28.9 25.4]
```

format (120,)

- 3) Déterminer la transposée de la matrice

```
# determinons la transposée de la matrice
mesures_5_capteurs_transpose = mesures_5_capteurs.T
print("Transposée de la matrice des mesures de 5 capteurs sur 24 heures:\n", mesures_5_capteurs_transpose)
print("format", mesures_5_capteurs_transpose.shape)
```

✓ 0.0s Python

Transposée de la matrice des mesures de 5 capteurs sur 24 heures:

```
[[23.7 21.6 20.2 21.8 26.1 27.9 26.1 28.1 21.2 26.6 29.7 29.2 23.9 25.4
 27.7 27.3 28.6 23.3 21.2 24.9 20.3 22.5 22.9 28.7]
 [29.5 20.6 29.7 23.  21.4 22.  21.7 23.  25.  23.1 27.8 20.9 22.7 21.4
 22.  27.7 26.2 27.3 27.1 25.2 26.4 24.1 21.6 28.  ]
 [27.3 28.7 28.3 25.2 22.9 25.1 20.7 21.  20.3 25.2 29.4 22.  28.3 28.
 20.1 20.7 23.3 26.4 27.6 24.3 23.1 27.6 29.3 21.9]
 [26.  26.  22.1 24.3 23.7 25.9 29.5 26.8 29.1 25.5 28.9 20.5 23.6 20.7
 28.2 23.6 20.6 28.9 25.6 20.3 25.1 22.3 28.1 28.9]
 [21.6 27.1 21.8 22.9 24.6 20.5 29.7 24.4 22.6 21.8 26.  23.3 22.8 29.9
 27.1 21.2 23.1 24.7 27.7 21.1 29.1 20.8 26.3 25.4]]
```

format (5, 24)

- 4) Extraire toutes les mesures du capteur 1 ;

```
# On extrait toutes les mesures du capteur 1
mesures_capteur_1 = mesures_5_capteurs[:, 0]
print("Mesures du capteur 1 sur 24 heures:\n", mesures_capteur_1)
```

✓ 0.0s Python

Mesures du capteur 1 sur 24 heures:
[23.7 21.6 20.2 21.8 26.1 27.9 26.1 28.1 21.2 26.6 29.7 29.2 23.9 25.4
27.7 27.3 28.6 23.3 21.2 24.9 20.3 22.5 22.9 28.7]

- 5) Extraire toutes les mesures du capteur 5 ;

```
# Ici on extrait toutes les mesures du capteur 5
mesures_capteur_5 = mesures_5_capteurs[:, 4]
print("Mesures du capteur 5 sur 24 heures:\n", mesures_capteur_5)
```

✓ 0.0s Python

Mesures du capteur 5 sur 24 heures:
[21.6 27.1 21.8 22.9 24.6 20.5 29.7 24.4 22.6 21.8 26. 23.3 22.8 29.9
27.1 21.2 23.1 24.7 27.7 21.1 29.1 20.8 26.3 25.4]

- 6) Extraire les mesures de tous les capteurs à 10h ;

```
# Les mesures de toutes les capteurs a 10h
mesures_capteurs_10h = mesures_5_capteurs[9]
print("Mesures des capteur 5 a 10 heures:\n", mesures_capteurs_10h)
```

✓ 0.0s Python

Mesures des capteur 5 a 10 heures:
[26.6 23.1 25.2 25.5 21.8]

- 7) Extraire les mesures des capteurs 2 à 4 entre 8h et 12h.

```
# les mesures des capteurs 2 a 4 entre 8h et 12h
mesures_capteurs_2_4_8h_12h = mesures_5_capteurs[6:12, 1:4]
print("les mesures des capteurs 2 a 4 entre 8h et 12h", mesures_capteurs_2_4_8h_12h)
```

✓ 0.0s Python

les mesures des capteurs 2 a 4 entre 8h et 12h [[21.7 20.7 29.5]
[23. 21. 26.8]
[25. 20.3 29.1]
[23.1 25.2 25.5]
[27.8 29.4 28.9]
[20.9 22. 20.5]]

Partie 7 – Concaténation

On suppose qu'on a un 2nd bâtiment avec 3 capteurs.

- 1) Créer aléatoirement la matrice représentant les températures mesurées par les 3 capteurs durant une journée

```
# Creons aleatoirement la matrice representant les temperatures mesurees par les 3 capteurs durant une journees
np.random.seed(42)
mesures_3_capteurs = np.random.uniform(15,37, (24,3)).round(1)
print(mesures_3_capteurs)
print("Format", mesures_3_capteurs.shape)
```

✓ 0.0s Python

[[23.2 35.9 31.1]
[28.2 18.4 18.4]
[16.3 34.1 28.2]
[30.6 15.5 36.3]
[33.3 19.7 19.]
[19. 21.7 26.5]

- 2) Déterminer de deux manières différentes la matrice qui fusionne les températures mesurées par les 8 capteurs durant une journée

```
# Premiere methode np.concatenate
matrice_8_capteurs_M1 = np.concatenate((mesures_5_capteurs, mesures_3_capteurs), axis=1)
print(matrice_8_capteurs_M1)
```

```
[[23.7 29.5 27.3 26.  21.6 23.2 35.9 31.1]
 [21.6 20.6 28.7 26.  27.1 28.2 18.4 18.4]
 [20.2 29.7 28.3 22.1 21.8 16.3 34.1 28.2]
 [21.8 23.  25.2 24.3 22.9 30.6 15.5 36.3]
 [26.1 21.4 22.9 23.7 24.6 33.3 19.7 19. ]
 [27.9 22.  25.1 25.9 20.5 19.  21.7 26.5]
 [26.1 21.7 20.7 29.5 29.7 24.5 21.4 28.5]
 [28.1 23.  21.  26.8 24.4 18.1 21.4 23.1]
 [21.2 25.  20.3 29.1 22.6 25.  32.3 19.4]
 [26.6 23.1 25.2 25.5 21.8 26.3 28.  16. ]
 [29.7 27.8 29.4 28.9 26.  28.4 18.8 16.4]
 [29.2 20.9 22.  20.5 23.3 35.9 36.2 32.8]
 [23.9 22.7 28.3 23.6 22.8 21.7 17.1 30.1]]
```

```
# deuxieme methode np.hstack
matrice_8_capteurs_M2 = np.hstack((mesures_5_capteurs, mesures_3_capteurs))
print(matrice_8_capteurs_M2)
```

```
[[23.7 29.5 27.3 26.  21.6 23.2 35.9 31.1]
 [21.6 20.6 28.7 26.  27.1 28.2 18.4 18.4]
 [20.2 29.7 28.3 22.1 21.8 16.3 34.1 28.2]
 [21.8 23.  25.2 24.3 22.9 30.6 15.5 36.3]
 [26.1 21.4 22.9 23.7 24.6 33.3 19.7 19. ]
 [27.9 22.  25.1 25.9 20.5 19.  21.7 26.5]
 [26.1 21.7 20.7 29.5 29.7 24.5 21.4 28.5]
 [28.1 23.  21.  26.8 24.4 18.1 21.4 23.1]
 [21.2 25.  20.3 29.1 22.6 25.  32.3 19.4]
 [26.6 23.1 25.2 25.5 21.8 26.3 28.  16. ]
 [29.7 27.8 29.4 28.9 26.  28.4 18.8 16.4]
 [29.2 20.9 22.  20.5 23.3 35.9 36.2 32.8]
 [23.9 22.7 28.3 23.6 22.8 21.7 17.1 30.1]]
```

Partie 8 : Calcul scientifique

Considérons le tableau représentant les 24 températures du 1^{er} bâtiment.

- 1) Pour les températures calculer le minimum, le maximum, la somme, la moyenne, la médiane, la variance et l'écart-type

```
# 1) Statistiques descriptives
t_min = temp_jour.min()
t_max = temp_jour.max()
t_sum = temp_jour.sum()
t_mean = temp_jour.mean()
t_median = np.median(temp_jour)
t_var = temp_jour.var()
t_std = temp_jour.std()

print(f"Minimum      : {t_min:.2f}")
print(f"Maximum       : {t_max:.2f}")
print(f"Somme           : {t_sum:.2f}")
print(f"Moyenne        : {t_mean:.2f}")
print(f"Médiane        : {t_median:.2f}")
print(f"Variance       : {t_var:.2f}")
print(f"Écart-type     : {t_std:.2f}")
```

```
Minimum      : 24.20
Maximum       : 34.70
Somme         : 692.10
Moyenne       : 28.84
Médiane       : 28.05
Variance      : 9.96
Écart-type    : 3.16
```

- 2) Standardiser le tableau des températures puis vérifier les valeurs de la moyenne et de l'écarttype du tableau standardisé

Formule de la standardisation

$$Z_i = \frac{X_i - \mu}{\sigma}$$

```
# standardisons le tableau des temperatures puis verifions les valeurs de la moyenne
temp_jour_standardiser = (temp_jour - t_mean)/t_std

#Verifions les valeurs de la moyenne et de l'ecart-type
print("Tableau standardisé :", temp_jour_standardiser.round(2))
print("Moyenne après standardisation :", round(temp_jour_standardiser.mean(), 10)) # ~0
print("Écart-type après standardisation :", round(temp_jour_standardiser.std(), 10))

Tableau standardisé : [-0.23  1.79  1.03  0.56 -0.99 -0.99 -1.34  1.48  0.56  0.94 -1.47  1.86
 1.38 -0.8  -0.9  -0.9  -0.49  0.3  -0.01 -0.52  0.59 -1.06 -0.52 -0.27]
Moyenne après standardisation : -0.0
Écart-type après standardisation : 1.0
```

3) Normaliser (Min-Max) le tableau des températures puis vérifier les valeurs du tableau normalisé

Formule de la normalisation Min-Max

$$X_{\text{normalisé}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

Les valeurs normalisées sont comprises entre 0 et 1 :

$$0 \leq X_{\text{normalisé}} \leq 1$$

```
# 3) Normalisation Min-Max : (x - min) / (max - min)
temp_jour_normalisees = (temp_jour - t_min) / (t_max - t_min)

print("Tableau normalisé :", temp_jour_normalisees.round(2))
print("Min après normalisation :", temp_jour_normalisees.min()) # 0
print("Max après normalisation :", temp_jour_normalisees.max()) # 1

Tableau normalisé : [0.37 0.98 0.75 0.61 0.14 0.14 0.04 0.89 0.61 0.72 0.  1.  0.86 0.2
 0.17 0.17 0.3  0.53 0.44 0.29 0.62 0.12 0.29 0.36]
Min après normalisation : 0.0
Max après normalisation : 1.0
```

4) Convertir le tableau initial en Fahrenheit

Formule de conversion Celsius → Fahrenheit

$$F = \frac{9}{5}C + 32$$

où C représente la température en degrés Celsius et F la température en degrés Fahrenheit.

```
# 4) Conversion en Fahrenheit : F = C * 9/5 + 32
temp_jour_fahrenheit = temp_jour * (9 / 5) + 32
print("Températures en Fahrenheit :", temp_jour_fahrenheit.round(1))

Températures en Fahrenheit : [82.6 94.1 89.8 87.1 78.3 78.3 76.3 92.3 87.1 89.2 75.6 94.5 91.8 79.3
 78.8 78.8 81.1 85.6 83.8 81.  87.3 77.9 81.  82.4]
```


Partie 9 : Broadcasting

- 1) Créer aléatoirement la matrice **temperatures_3x3** représentant les températures mesurées à 10h, 11h et 12h dans le 2nd bâtiment

```
np.random.seed(42)
temperatures_3x3 = mesures_3_capteurs[9:12]
temperatures_3x3
```

array([[26.3, 28. , 16.],
 [28.4, 18.8, 16.4],
 [35.9, 36.2, 32.8]])

- 2) Supposons que les valeurs de calibration des capteurs soient stockées dans le tableau **correction = np.array([1, -0.5, 0.2])**

```
# Valeurs de calibration des 3 capteurs
correction = np.array([1, -0.5, 0.2])
```

Appliquer automatiquement les corrections pour trouver les mesures corrigées
mesures_corrigees = temperatures_3x3 + correction

```
correction

# Application des corrections par Broadcasting
mesures_corrigees = temperatures_3x3 + correction

print("\nCorrections :")
print(correction)

print("\nMesures corrigées :")
print(mesures_corrigees)
```

Corrections :
[1. -0.5 0.2]

Mesures corrigées :
[[27.3 27.5 16.2]
 [29.4 18.3 16.6]
 [36.9 35.7 33.]]